

# Auditoría pedagógica de la fase 2

## Alcance y criterio

Se han auditado individualmente los veinte módulos indicados: cinco de fundamentos, dos de metodología, ocho de seguridad web y cinco de Linux. Esta auditoría no modifica su contenido ni cambia su estado editorial.

La valoración mide si el material enseña a razonar, construir, diagnosticar, adaptar y transferir. No premia la longitud ni la cantidad de comandos. Escala usada: 4 sólido, 3 adecuado, 2 parcial, 1 débil, 0 ausente. N/A indica que la construcción de payloads no es el objetivo principal del módulo. Prioridad: P0 crítica, P1 alta, P2 media.

## Resultado ejecutivo

La base conceptual es buena en fundamentos y metodología, especialmente en separación de capas, flujo source-sink, controles negativos y distinción entre shell y argv. El problema principal está en los módulos técnicos: conservan teoría y comandos útiles, pero su capa pedagógica fue materializada mediante una plantilla común.

En los trece módulos técnicos de web y Linux revisados se repiten literalmente el objetivo, el modelo mental, las seis etapas de construcción, la tabla de diagnóstico, el caso guiado, el caso de adaptación y los cuatro ejercicios. Ninguno contiene todavía un escenario A/B/C/D completo, un falso positivo desarrollado, una resolución razonada específica ni la anatomía obligatoria completa de un payload. La transferencia se menciona, pero rara vez se practica.

También se detectaron riesgos técnicos que deben corregirse al desarrollar los lotes: se confunden indicio y confirmación, se omiten dependencias de motor o configuración y algunos ejemplos presuponen una shell, separación en varios elementos de argv, conservación de privilegios o interpretación de ficheros sin haberlo demostrado.

## Tabla de priorización

| Módulo                              | Fundamentos | Construcción | Adaptación | Ejercicios | Problema principal                                                                                                           | Prioridad |
|-------------------------------------|-------------|--------------|------------|------------|------------------------------------------------------------------------------------------------------------------------------|-----------|
| http_y_sesiones.md                  | 4           | N/A          | 2          | 1          | Buen mapa de capas, sin escenarios que obliguen a atribuir una diferencia a transporte, sesión, autenticación o autorización | P2        |
| redes_dns_y_urls.md                 | 4           | N/A          | 2          | 0          | Explica parsing diferencial, pero no enseña a demostrar quién resuelve, conecta o selecciona el vhost                        | P2        |
| encoding_normalizacion_y_parsers.md | 4           | 2            | 3          | 0          | Modelo excelente pero abstracto; faltan trazas comparadas de decodificación y falsos diagnósticos                            | P1        |
| linux_procesos_permisos_y_shell.md  | 4           | 2            | 3          | 0          | Distingue shell y argv, pero no hace construir experimentos que identifiquen cuál existe                                     | P1        |
| bases_de_datos_y_flujo_de_datos.md  | 4           | 2            | 2          | 0          | Buen source-sink; falta reconstruir consultas y distinguir contexto textual, numérico e identificadores                      | P1        |

| Módulo                                   | Fundamentos | Construcción | Adaptación | Ejercicios | Problema principal                                                                                                | Prioridad |
|------------------------------------------|-------------|--------------|------------|------------|-------------------------------------------------------------------------------------------------------------------|-----------|
| metodologia_de_laboratorio.md            | 4           | N/A          | 3          | 1          | Método sólido sin casos completos de decisión, descarte y reapertura de hipótesis                                 | P1        |
| construccion_y_adaptacion_de_payloads.md | 4           | 3            | 3          | 1          | Buena base; casos comprimidos, anatomía incompleta y ausencia del árbol y cinco desarrollos exigidos              | P0        |
| sqli.md                                  | 2           | 2            | 1          | 1          | Lista extensa de cadenas que salta entre contextos y motores; errores y demoras se sobrevaloran como confirmación | P0        |
| lfi.md                                   | 2           | 2            | 1          | 1          | Mezcla traversal, lectura, include e interpretación; no diagnostica sufijos, raíz, wrappers ni permisos           | P0        |
| ssrf.md                                  | 3           | 2            | 2          | 1          | Tiene una progresión útil, pero no desarrolla parser, resolución, redirects, cliente HTTP ni falsos positivos     | P0        |

| Módulo                   | Fundamentos | Construcción | Adaptación | Ejercicios | Problema principal                                                                                                 | Prioridad |
|--------------------------|-------------|--------------|------------|------------|--------------------------------------------------------------------------------------------------------------------|-----------|
| command_injection.md     | 2           | 2            | 1          | 1          | Presupone shell y output; no distingue concatenación, argv, canal blind y restricciones del entorno                | P0        |
| argument_injection.md    | 2           | 1            | 1          | 1          | El ejemplo depende de que un wrapper divida texto en varios argumentos, condición que no se demuestra              | P0        |
| file_upload.md           | 2           | 1            | 1          | 1          | El título y la prueba saltan de almacenamiento a ejecución; validación y handler se explican de forma absoluta     | P0        |
| mfa_otp_bypass.md        | 2           | 1            | 1          | 1          | Reduce MFA a <code>is_verified=true</code> ; faltan estado, binding, sesión, replay, rate limit y falsos positivos | P0        |
| auth_session_security.md | 1           | 1            | 1          | 1          | Demasiado superficial para distinguir autenticación, sesión, expiración, revocación y autorización                 | P0        |

| Módulo                | Fundamentos | Construcción | Adaptación | Ejercicios | Problema principal                                                                                                 | Prioridad |
|-----------------------|-------------|--------------|------------|------------|--------------------------------------------------------------------------------------------------------------------|-----------|
| enum_privesc_linux.md | 2           | 1            | 1          | 1          | Enseña a ejecutar herramientas y priorizar colores, no a convertir un hallazgo en relación explotable o descartada | P1        |
| sudo_abuse.md         | 2           | 2            | 1          | 1          | Flujo mecánico sudo -l -> GTFOBins -> shell; no analiza regla, argumentos, entorno, versión ni capacidad mínima    | P1        |
| suid.md               | 2           | 1            | 1          | 1          | Generaliza el efecto SUID y omite nosuid, no_new_privs, scripts y pérdida de privilegios en intérpretes            | P0        |
| cron_abuse.md         | 2           | 1            | 1          | 1          | Confunde fichero escribible con escalada confirmada; faltan identidad, entorno, frecuencia y evidencia inocua      | P1        |
| path_hijacking.md     | 2           | 1            | 1          | 1          | strings solo aporta una pista y el ejemplo no demuestra PATH heredado ni conservación de privilegios               | P0        |

# Diagnóstico individual

## Fundamentos

### #### http\_y\_sesiones.md

- Modelo mental: sólido; ordena proxy, parser HTTP, router, autenticación, parser de cuerpo, lógica y autorización.
- Casos guiados y adaptación: la práctica final propone cambiar variables por separado, pero no ofrece respuestas concretas, hipótesis rivales ni resolución.
- Repetición: baja; contenido propio y conciso.
- Transferencia: adecuada como prerrequisito, pero no se evalúa la misma entrada en query, cookie, JSON y multipart ni la diferencia autenticación/autorización.

### #### redes\_dns\_y\_urls.md

- Modelo mental: adecuado; separa URL, DNS, transporte, TLS, SNI y Host.
- Casos guiados y adaptación: hay ejemplos puntuales de puertos y parsing diferencial, no un escenario reproducible con controles.
- Repetición: baja.
- Transferencia: potencial alto hacia SSRF y vhosts; falta practicar quién realiza cada paso y qué observación distingue DNS, conexión y aplicación.

### #### encoding\_normalizacion\_y\_parsers.md

- Modelo mental: sólido y directamente alineado con la fase 2.
- Construcción y adaptación: el ejemplo %2e%2e%2fconfig y el orden de procesamiento son útiles, pero no se reconstruyen dos pipelines alternativos a partir de respuestas reales.
- Casos y ejercicios: ausentes.
- Transferencia: alta en teoría; debe conectarse explícitamente a URL, rutas, JSON, SQL y shell mediante casos contrastados.

### #### linux\_procesos\_permisos\_y\_shell.md

- Modelo mental: sólido; explica proceso, credenciales, shell, operadores, argumentos y opciones.
- Construcción y adaptación: parcial; no obliga a predecir argv, quoting, expansiones y redirecciones en escenarios concretos.
- Casos y ejercicios: ausentes.
- Transferencia: alta hacia command injection, argument injection y privesc, todavía sin práctica discriminatoria.

### #### bases\_de\_datos\_y\_flujo\_de\_datos.md

- Modelo mental: sólido; source, transformación, validación y sink están bien definidos.
- Construcción: parcial; la prueba mínima se describe, pero no se deriva desde la consulta original ni compara contextos.
- Casos y ejercicios: ausentes.
- Transferencia: adecuada a SQLi y a otros sinks; falta exigir que el alumno reconstruya el flujo con evidencia incompleta.

## Metodología

### #### metodologia\_de\_laboratorio.md

- Modelo mental: sólido; incluye niveles de certeza, controles y criterios para abandonar hipótesis.
- Casos guiados: ausentes; las tablas están vacías y no muestran decisiones sucesivas.
- Adaptación: adecuada como clasificación general, pero no enseña cómo elegir el experimento que separa dos causas en técnicas distintas.

- Repetición y transferencia: baja repetición interna y alta transferencia potencial. Necesita dos o tres casos completos, incluido uno cuya hipótesis inicial sea falsa.

#### #### construccion\_y\_adaptacion\_de\_payloads.md

- Modelo mental: sólido y central para el curso.
- Construcción: adecuada; explica contexto, cierre, serialización y canales. La anatomía actual omite varios campos exigidos en fase 2 y los diez casos son resúmenes, no desarrollos.
- Adaptación y diagnóstico: adecuados en general, insuficientes para síntomas específicos. Falta el árbol solicitado, comparaciones correcto/incorrecto, capas múltiples y cinco casos completos.
- Casos, ejercicios y transferencia: los ejemplos cubren muchas técnicas y favorecen transferencia conceptual, pero no incluyen preguntas, controles, pistas ni resolución paso a paso.

## Seguridad web

#### #### sqli.md

- Fundamentos y modelo mental: la causa básica es reconocible, pero se afirma una causa compartida con LFI sin precisar sinks distintos y el modelo es la plantilla genérica.
- Construcción: diecisiete “capas” forman una lista de payloads y herramientas; cambian contexto, motor, canal y objetivo sin derivación completa. Comillas simples alrededor de comandos con \$URL introducen además riesgo de transporte local incorrecto.
- Adaptación, casos y ejercicios: genéricos y clonados. No hay Caso A-D, diagnóstico por síntomas ni falso positivo.
- Transferencia: débil; se memoriza la secuencia error -> UNION -> catálogo -> dump más que la reconstrucción de consulta y oráculo.

#### #### lfi.md

- Fundamentos y modelo mental: presenta un ejemplo PHP útil, pero trata como equivalentes traversal, lectura e inclusión y sugiere ejecución sin demostrar el sink.
- Construcción: dos payloads con explicación parcial; no se documentan base, profundidad, sufijo añadido, wrapper habilitado, identidad ni control negativo.
- Adaptación, casos y ejercicios: bloques genéricos idénticos al resto.
- Transferencia: débil; no se contrasta con path traversal, descarga legítima, error de plantilla ni fichero inaccesible.

#### #### ssrf.md

- Fundamentos y modelo mental: buenos para reconocer la primitiva servidor -> destino, aunque el modelo específico no está dibujado.
- Construcción: listener, callback y loopback forman una progresión razonable. 127.1 y otras representaciones se presentan casi como universales, sin vincularlas al parser y normalizador observados.
- Adaptación y diagnóstico: se nombran filtros, redirects y herramienta externa, pero la tabla no contiene síntomas SSRF reales.
- Casos y transferencia: no hay escenarios concretos; la superficie enumera avatar, webhook, importador y PDF sin convertirlos en tareas de reconocimiento.

#### #### command\_injection.md

- Fundamentos: mezcla APIs con y sin shell al agrupar system, exec y popen; el significado de ; depende de la invocación real.
- Construcción: un único ejemplo JSON ya presupone sh -c, salida visible y entorno Unix. Falta derivar cierre, separador, comando mínimo, serialización y control.
- Adaptación y diagnóstico: no hay síntomas propios como carácter literal, cambio de error, ejecución blind consistente o fallo de red de una shell.
- Transferencia: débil y sin comparación experimental con argument injection.

#### #### argument\_injection.md

- Fundamentos: distingue correctamente shell de parser de opciones, pero el texto oscila entre “entrada como argumento” y “línea concatenada”. Un espacio solo crea otro elemento de argv si existe una capa de splitting.
- Construcción: un único payload de curl presupone splitting, soporte de esquema y devolución de ambos resultados.
- Adaptación, casos y ejercicios: genéricos; no se reconoce versión ni se compara opción, operando y --.
- Transferencia: potencial alto a curl, tar, conversores y wrappers; actualmente se enseña como catálogo de capacidades.

#### #### file\_upload.md

- Fundamentos: identifica capas relevantes, pero el título “a shell” y varias frases hacen parecer inevitable la ejecución. Almacenar, recuperar e interpretar deben ser primitivas separadas.
- Construcción: el ejemplo modifica extensión y Content-Type a la vez, por lo que no aísla qué control se aplica; tampoco define bytes, nombre final, ruta o handler.
- Adaptación y diagnóstico: no hay síntomas de renombrado, serving estático, MIME, magic bytes, antivirus, almacenamiento externo o extensión no interpretada.
- Transferencia: débil; faltan avatar, adjunto, importación y almacenamiento de objetos como escenarios distintos.

#### #### mfa\_otp\_bypass.md

- Fundamentos: explica bien que el servidor debe mantener el estado, pero reduce el espacio de fallos a una bandera controlada por cliente.
- Construcción: un único multipart presupone nombre, endpoint y transición. No modela sesión pre-MFA, OTP ligado a usuario, estado posterior ni acceso directo al recurso.
- Adaptación y diagnóstico: ausentes en términos específicos; no diferencia OTP inválido, expirado, reutilizado, de otro usuario o sesión.
- Transferencia: débil hacia recuperación de cuenta, cambio de factor, “remember device” y flujos API.

#### #### auth\_session\_security.md

- Fundamentos y modelo mental: insuficientes; la teoría ocupa dos frases y el modelo genérico no representa estados, credenciales, rotación, expiración ni recursos.
- Construcción: las dos llamadas a /api/me solo identifican si un token autentica; no demuestran seguridad o vulnerabilidad.
- Adaptación, casos y ejercicios: genéricos y sin síntomas específicos.
- Transferencia: baja; no diferencia bypass de autenticación, bypass de autorización, sesión inválida, caché ni endpoint público.

## Linux

#### #### enum\_privesc\_linux.md

- Fundamentos: ofrece un orden operativo, pero trata colores de linneas y NOPASSWD como evidencia demasiado cercana a confirmación.
- Modelo mental: la plantilla source-sink no representa bien sujeto, objeto, operación, identidad efectiva y frontera escribible.
- Construcción y adaptación: se ejecutan herramientas; no se construye una hipótesis desde su salida ni se enseña a descartar falsos positivos.
- Casos, ejercicios y transferencia: genéricos; falta una salida mixta con pistas reales y ruido.

#### #### sudo\_abuse.md

- Fundamentos: útiles pero dependientes de la receta de GTF0Bins; no toda regla NOPASSWD implica escape.
- Construcción: sudo -l y find -exec no forman una adaptación; falta leer usuario destino, host, comando, argumentos permitidos, tags y entorno.

- Casos y diagnóstico: genéricos; no contempla argumentos fijos, ruta distinta, versión, TTY, contraseña, NOEXEC o capacidad limitada.
- Transferencia: débil; debería trasladar el mismo análisis a lectura, escritura y ejecución sin exigir shell root.

#### #### suid.md

- Fundamentos: correctos a alto nivel, pero demasiado absolutos. Linux puede ignorar SUID por nosuid, no\_new\_privs o tracing; lo ignora en scripts, y un intérprete puede soltar privilegios.
- Construcción: solo enumera con find; delega la explotación a una receta externa y no modela UID real, efectivo y guardado.
- Adaptación y diagnóstico: genéricos; faltan propietario no root, mount nosuid, binario que baja privilegios y función no alcanzable.
- Transferencia: baja hacia binarios a medida y capacidades parciales.

#### #### cron\_abuse.md

- Fundamentos: reconoce periodicidad, identidad y escritura, pero mezcla script escribible, wildcard y PATH sin separarlos.
- Construcción: enumera crontab y permisos; salta después a reverse shell o Bash SUID sin prueba inocua intermedia.
- Adaptación y diagnóstico: no cubre entorno mínimo, working directory, shell, frecuencia, propietario, permisos de directorio ni unidad systemd alternativa.
- Transferencia: parcial hacia otros ejecutores programados; no hay escenario donde la tarea observada sea de otro usuario o no se ejecute.

#### #### path\_hijacking.md

- Fundamentos: explica la condición central, pero strings no confirma una llamada y un proceso privilegiado puede fijar o sanear PATH.
- Construcción: el payload crea varios cambios simultáneos y /bin/bash puede perder privilegios si no se invoca de la forma adecuada; falta un marcador inocuo.
- Adaptación y diagnóstico: no contempla comando cacheado, ruta absoluta, secure\_path, entorno no heredado, permisos o identidad efectiva.
- Transferencia: parcial entre SUID, sudo, cron y servicios, sin ejercicios que obliguen a distinguirlos.

## Patrones mecánicos detectados

En los trece módulos técnicos se repiten exactamente, entre otros, estos bloques:

- objetivo y prerequisites;
- modelo Entrada controlada -> transformaciones -> validación -> componente final;
- seis pasos de "Construcción progresiva del payload";
- cinco preguntas de autoevaluación;
- párrafos de variaciones, filtros, control negativo, escalado, mitigación y certificaciones;
- tabla genérica de cuatro síntomas;
- caso guiado y caso de adaptación sin escenario;
- cuatro ejercicios idénticos;
- chuleta operativa y navegación con texto pendiente.

La repetición no es problemática en plantillas de referencia o criterios comunes. Sí lo es aquí porque ocupa las secciones que deberían contener práctica específica y crea una falsa sensación de cobertura. Debe conservarse la metodología común en los documentos de metodología y reemplazarse en cada técnica por evidencia, parser, restricciones y decisiones propias.

## Comparaciones transversales ausentes

| Comparación                                   | Estado actual                                   | Necesidad pedagógica                                                            |
|-----------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------|
| Command injection vs argument injection       | Diferencia declarada, no demostrada             | Comparar shell grammar, splitting, argv, opciones y errores con el mismo input  |
| Path traversal vs LFI                         | Mezcladas en fundamentos y escalado             | Separar apertura, lectura, include, interpretación y wrappers                   |
| SSRF vs open redirect vs petición client-side | SSRF menciona que la URL podría ser client-side | Demostrar quién inicia la conexión mediante listener, navegador y respuesta 3xx |
| Authentication bypass vs authorization bypass | Definición breve en fundamentos HTTP            | Modelar identidad, estado, acción y objeto protegido con controles cruzados     |
| File upload vs ejecución                      | Se expresa como un único objetivo               | Separar aceptación, persistencia, localización, recuperación e interpretación   |

## Lista ordenada de mejoras

1. Corregir el estándar central de payloads. Ampliar `construccion_y_adaptacion_de_payloads.md` sin reescribirlo: árbol de diagnóstico, anatomía completa exigida, cinco casos desarrollados, ejemplos correctos/incorrectos, capas múltiples y conclusiones iniciales falsas.
2. Rehacer SQLi y LFI desde el contexto, no desde listas. Construir consultas y rutas resultantes; separar motores, canales, lectura e inclusión; añadir casos A-D y diagnóstico específico.
3. Crear el bloque comparativo shell/argv. Coordinar SSRF, command injection y argument injection con un mismo escenario de wrapper HTTP y pruebas que distingan biblioteca, CLI con argv y `sh -c`.
4. Separar upload de ejecución. Convertir cada capa en una primitiva verificable y usar un control por vez: metadatos, bytes, nombre, almacenamiento, serving y handler.
5. Modelar autenticación y MFA como máquinas de estados. Incluir sesión preautenticada, transición MFA, rotación, binding, revocación y autorización del recurso; ampliar más allá de una bandera.
6. Sustituir el modelo genérico de Linux. Usar sujeto -> capacidad controlada -> proceso privilegiado -> resolución/ejecución -> efecto, con UID real/efectivo y controles inocuos.
7. Añadir casos A-D a cada módulo técnico. Cada caso debe tener escenario, preguntas, resolución observación -> hipótesis -> prueba -> resultado -> nueva hipótesis y criterio de descarte.
8. Crear diagnósticos por síntoma. Reemplazar las cuatro filas clonadas por salidas reales de SQL, PHP, curl, shell, sudo, permisos, cron y PATH.
9. Convertir transferencia declarada en ejercicios. Reutilizar la primitiva en funcionalidades distintas sin nombrar la vulnerabilidad y exigir mapa entrada -> transformaciones -> parser -> sink.
10. Desarrollar primero los fundamentos que desbloquean varios lotes. Añadir casos cortos a encoding/parsers, shell/argv, bases de datos y metodología; evitar convertirlos en enciclopedias.
11. Corregir referencias editoriales obsoletas. `http_y_sesiones.md` dice que el cuaderno estará disponible y los trece módulos afirman que ejercicios y chuletas "se enlazarán" aunque ya existen.
12. Mantener estados conservadores. Los cinco fundamentos y dos documentos de metodología ya figuran como revisado, pero solo deben conservarlo tras la revisión técnica/pedagógica del lote; los trece técnicos deben permanecer borrador hasta cumplir y validar sus casos.

## Criterios de aceptación para los lotes siguientes

Un módulo técnico podrá considerarse revisado cuando:

1. contenga casos A, B, C y D específicos y no intercambiables con otra técnica;
2. al menos un payload completo use todos los campos obligatorios de anatomía;
3. el Caso B diagnostique la causa antes de adaptar;
4. el Caso C oculte el nombre de la vulnerabilidad y exija reconocer la primitiva;
5. el Caso D incluya una prueba que descarte la hipótesis;
6. el diagnóstico contenga síntomas propios del parser o componente;
7. los ejercicios obliguen a elegir experimentos, no a repetir definiciones;
8. las afirmaciones dependientes de motor, herramienta, versión o configuración estén delimitadas y respaldadas;
9. no conserve como caso o ejercicio ninguno de los bloques genéricos detectados;
10. pase validación de metadatos, enlaces, cercas, payloads y PDF, además de revisión visual representativa.

## Notas de verificación técnica

- La CLI de curl admite varias URL, pero el payload de argument injection solo las produce si la aplicación construye realmente varios elementos de la línea de comandos; debe demostrarse la capa de splitting. Véase [documentación oficial de curl sobre múltiples URL](#).
- En Linux, SUID modifica el UID efectivo al ejecutar un binario, pero puede ignorarse con no\_new\_privs, montajes nosuid o tracing; además, Linux ignora SUID/SGID en scripts. Véase [execve\(2\)](#).
- Bash restablece por defecto el UID efectivo al real cuando ambos difieren, salvo modo privilegiado. Por eso “lanzar /bin/bash” no basta como explicación universal de conservación de privilegios. Véase [GNU Bash: invocación con UID real y efectivo distintos](#).

## Estado al cerrar la auditoría

- Auditoría completada y guardada.
- Ningún módulo del alcance fue reescrito.
- No se modificaron Windows, Active Directory, redes, ejercicios, solucionarios, progreso ni PDF.
- Próximo trabajo previsto: Lote 1, fundamentos y metodología, seguido de su validación y registro.